PROJECT WIKI

Yumeniwa

An otaku media tracker, designed and built from scratch

Chaitanya Shinde
Built 30 October 2026

CONTENTS

1 The problem and who it's for 3
2 yumeniwa : one single stop leisure tracker for otakus 4
2.1 Problem : You don't own anything and its unreliable 4
2.2 Stance , Status and Future scope 4
2.3 Current limitations and gaps still need to understand 5
3 Features 6
3.1 planned features for initial rollout atleast 6
4 Entities and relationships 7
4.1 Entities involved their relations and overall data involved in application 7
5 Main flows 8
5.1 base flow 8
5.2 for any media item 8
5.3 offline only but later network connects 8
5.4 network exists but is extremely flaky 9
5.5 state maintainence 9
6 API surface 10
6.1 typical routes 10
7 Non-functional needs 11
8 Research: How Hard Is "Bring Your Own Provider"? 12
8.1 What BYO can mean: three levels 12
8.2 The six hard parts 12
8.3 How others did it 13
8.4 Feasibility at a glance 13

CHAPTER 1

The problem and who it’s for

CHAPTER 2

yumeniwa : one single stop leisure tracker for otakus

2.1 Problem : You don't own anything and its unreliable

goal in one sentence

Let user be in control of what they want how they want , for otakus let them track all their likes in one single place rather than letting it spread over

- **Dependance on one provider hurts reliability** : Many of tracking apps for otaku face one problem : they are dependent on one singleservice provider , like anilist or myanimelist (mention outage of anilist we had expereinced yesterday) , our solves this by allowing drive connectors , and persistent offline backups done frequently , so that data is always readily stored securedly in case if even one of service goes down , (diff based backups are encouraged or Write replace when new snapshot is ready we need to think about a decent snapshot mechanism for this)
- **Cannot track various media + niche** : one provider say anilist can only offer services to track information about anime , manga at best , but what about otakus who like visual novels games well , what about them , our solution : **aggregation** of providers for fetching stuff , with **bring your own providers** (this will not only need contract definations but also need very strong adapter based design to plug n play standard default aggregators we will ship + providers users wana plug in (to be honest can keep it in version v2) , also how hard is creating a custom extension for your own application) , (need to explain users about api limitations though like which provider has what rules we need to point them towards that , we can ensure by clarifying what each provideroffers what we can do with them)
- **Bidirectional focused (synced across providers only if they allow through api access)** : ensures data is consistent across local , cloud drive , providers , seammlessly ,

2.2 Stance , Status and Future scope

above are main problems faced by me atleast and what i wish to solve further improvmenets and future scopes could be

- add future socialization features: enabling users to create and engage in social forums and such , let users write extremely rich feedbacks and so on
- extremely robust user taste analyzer , detailed analytics and building user profile about their tastes , adding genn ai and ml ai stuff so that a depth detailed recommendations (based on category is showccased)
- potentially shipped with media viewers for anime manga lightnovels user can add their fav providers of actual content and watch and track at same time

2.3 Current limitations and gaps still need to understand

- u cannot fully really on any provider no matter what ? give user the ability to define his own entries manual as well ? these wont be considered for syncing though amongst providers , this niche entry lives seperately saved still in custom data and drive and what not
- most obvious and important is to enforce a single adaptder contract working for any kind of provider user may bring since version 1 scope is reliable providers shipped + only rerliable providers add this gives us room but actual thing using **ANY** provider you being is still tricky , cannot know ahead of time what to display , hmmmm still again not sure if this counts as a major risk / limitation
- still havent singled out reliable providers
- need to determine feasibility of setting up consistent pipeline and custom extension file or format or such a mechanism to ensure data is consistently maintained
- conflicts can arise , say anilist has one anime as watching , and user has added mal as provider in that due to user migrating from mal to anilist user has that same anime as planning in their (solution isolation : entry existing on most latest platform is picked as default (we can also let user know that this is choosen is it correct aggregation conflict exists) or let user solve conflicts by highlighting them)
- initially limited to just anime , manga , lightnovel , visual novel , games

(many other things like webnovels , doujinshi , live action adaptions , vocaloid etc etc may fall out of scope)

CHAPTER 3

Features

3.1 planned features for initial rollout atleast

- tracks across verified providers
- maintains consistency across all providers and local and drive backups

(need a very strong idea to solve above)

- user should be in control and their interest should be seemlessly tracked

CHAPTER 4

Entities and relationships

4.1 Entities involved their relations and overall data involved in application

4.1.0.1 majorly only user and the application itself are major two surfaces theres no such thing as role based accesses, admins etc none of that just user who is facing application thats it

- user : trivial with all relevant fields , user auth and all that holds great importance , this is a full fledged aggregation tracker , building and maintaining user profile is of importance to us , proper standard to maintain here
- media entities (further categorized into sub entities) (notice the overlap between all media types some fields are gonna be common only fields depending on type will be changed slightly or reworded at best) (again solved the question of mongo vs psql a shame though i wanted to try out postgre too)
- anime : with all of its relevant fields
- manga : with relevant fields
- visual novel : with relevant fields
- light novels : with relevant fields
- game : with relevant fields
- webnovel if possible : with relevant fields

CHAPTER 5

Main flows

5.1 base flow

- standard onboarding :
  - user auths properly
  - our app gives tour and guide on how to use
  - default aggregators option to choose is presented and disabled , option to connnect to drive and or local extension persistent backup setting , frequencies , how and where to save more like drive , etc
- user starts using the application normally , auth related stuff and all profile should be maintiaiined and open export options to maintain personal backups aswell

5.2 for any media item

- a CRUD operation associated with it
- associated thing tagged and identified by provider , type ,
- hot updates done to that only agregator
- handling conflict is important aspect by heuristic treat latest entry in whatever aggregator as source of truth , also highlight user action for that media user has to clarify later which aggregator's version to be treated as true
- true version further processed onwards
- diff based dispatcher based updates done to other providers and local and cloud copy

IN THIS MANNER A CENTRAL STORE IS MAINTAINED BY OUR APP , OTHER AGGREGATORS ARE UPDATED FURTHER LATER FROM ACCORDING TO THAT

IN SIMPLE WORDS : INPUT ALL DATA → RESOLVE CONFLICTS CONFIRM → THIS FORMS OUR SOURCE OF TRUTH → TREAT EVERYTHING ACCORDING TO THIS SOURCE OF TRUTH

5.3 offline only but later network connects

- queuing of incremental updates and dispatch them to online services later when network connection is available
- offline version which app holds is always live , this is source of truth others aggregators and cloud sync from this reality when network later reconnects
- this is intention behind between **giving user more control and true reliability is what u have constantly locally**

5.4 network exists but is extremely flaky

recommended to handle like above , some types be exempt for this , since our main

5.5 state maintainence

- diff based data maintained or write entire copy first then replace mechanism , first one ideally preffered

CHAPTER 6

API surface

6.1 typical routes

- user essential routes of auth , refresh auth , profile
- crud for media items , can be of type create-media -> type itself identifies it
- dedicated sync everything route , and for locally saved result

CHAPTER 7

Non-functional needs

- need to have a smart offline detection , and queueing updates to whatever things concerned be saved in queue , when internet is returned , sync immedaitely with all relevant providers (smart sync will also tag whomsoever does the update concerned , for eg : something aggregated fromm anilist , means no need to include a mal api call , note which providers had update and just update those only) , persistent local and drive saves good too
- rate limits of each provider to behandled by waits
- capstone so all knowledge relevant from all coursework even cybsec will be used

x further issues can only be known as i keep on testing it and identifyinng such caveats

CHAPTER 8

Research: How Hard Is "Bring Your Own Provider"?

RESEARCH GUIDE · WRITTEN BY CLAUDE

Facts and pointers gathered for me to study. It informs my design; it does not make my decisions. Researched 23 October 2026. Provider behaviour was checked with live requests from this laptop that day.

8.1 What BYO can mean: three levels

"Bring your own provider" is not one feature. It is a ladder, and each rung costs much more than the one below it.

Level	What the user brings	What Yumeniwa must already contain
1 · Configure	Credentials or a base URL for a provider **type** Yumeniwa already knows (their own AniList account, a self-hosted instance)	One adapter per provider type, written by you
2 · Describe	A file (JSON or similar) that describes an unknown API: endpoints, auth style, how its fields map onto Yumeniwa's fields	A generic engine that reads such files and a format (a "contract") that can express every API you want to support
3 · Code	A plugin: real code that talks to the provider	A plugin API, a loader, and a sandbox, because the code is untrusted

8.2 The six hard parts

1. Every provider authenticates differently

AniList and MyAnimeList use OAuth 2.0; VNDB uses a personal API token with permissions (`listread`, `listwrite`) (VNDB docs) others use API keys or nothing. A level-2 file must be able to describe all of these, including how tokens refresh.

2. Browsers block some providers (CORS)

An offline-first PWA wants to call providers from the browser. Whether it can depends on each provider's CORS headers. Preflight requests sent from origin `http://localhost:5173` on 23 Oct:

Provider	Result
AniList	Allowed: `Access-Control-Allow-Origin: *`
VNDB	Allowed: echoes the requesting origin, allows `Authorization`
MyAnimeList API	**Blocked**: preflight answered `405`, no CORS headers
Kitsu, MangaUpdates	Unclear: the preflight got `403`; test again with a real request

A provider that blocks browsers can only be reached through a server that forwards the request (a relay). So BYO decides part of what your Express backend is for.

3. A relay that calls user-supplied URLs is a security hole

If users can make your server call **any** URL, an attacker can point it at internal addresses. This attack is **server-side request forgery** (SSRF); read the OWASP SSRF prevention cheat sheet and KnoTex Ch 19 (SSRF).

4. Running someone else’s code needs a real sandbox

Node’s documentation states plainly: “The `node:vm` module is not a security mechanism” (Node docs). In the browser, isolation comes from Web Workers and sandboxed iframes (MDN: iframe `sandbox`). Level 3 is a security project of its own.

5. Providers disagree on meaning, not just format

Status sets differ (AniList has `REPEATING` and `PAUSED`; MyAnimeList has `on_hold`; VNDB uses labels), and score scales differ. A contract has to say what happens to a value that has no equivalent on the other side: this is **lossy mapping**. Concepts to read: **canonical data model** and the anti-corruption layer pattern.

6. Limits and maintenance

Each provider has its own rate limit (AniList 90 requests per minute, temporarily 30; VNDB 200 requests per 5 minutes), pagination and error format. And when a provider changes its API, every adapter or description file for it breaks: someone has to maintain them.

8.3 How others did it

Project	Approach
Mihon trackers	Six trackers built into the app (level 1); sync is one-way, app to tracker, and queued while offline
Mihon extensions	Level 3 for **content sources**: installable code packages from separate repositories, with trust warnings to the user
Home Assistant integrations	Level 3 with a strict contract: every integration ships a manifest, and a published quality scale grades them
Yamtrack, Ryot	No BYO: a fixed set of providers plus imports

8.4 Feasibility at a glance

- **Level 1** is ordinary adapter work: one module per provider behind one interface. Feasible from day one.
- **Level 2** needs a contract expressive enough for auth, pagination, rate limits and field mapping. Feasible, but designing that format well is a project on its own; study how others describe APIs (for example the OpenAPI specification) before inventing one.
- **Level 3** adds sandboxing and SSRF defences. High risk for a first version.

> **OPEN QUESTION** **Questions for me to decide**
>
> Which level does v1 need? Which providers ship built in? Can a provider that blocks CORS still work offline-first, and what does the relay mean for the “no single point of failure” goal?